Design Discussion

The scene that I chose was a shelf on my bookshelf, with five main objects to fill the space. These objects were a concert lightstick, a notebook, a candle, a cat plush, and a camera. These objects vary in their complexity, from the notebook, made from boxes to things like the camera and the lightstick, which combine cylinders, tapered cylinders, boxes, and more meshes. Having a diverse group of different objects allowed me to practice different materials, shapes, scaling, and rotations while filling the shelf space. Certain functionality, such as the blue light coming from the lightstick allowed me to learn how to position light and make sure that the blue light was not washed out by main/fill lights. Having objects with an array of difficulties allowed me to focus on some of the more complex ones while still having simple objects to fill the space. Creating a simple object like the candle allowed me to have more time to create interesting logic for the camera's cylinder rotations. Because the camera lens is laid on its side with a 90-degree X rotation, I apply a Z rotation first, using the camera's turn angle, so the lens turns with the rest of the camera. I might not have had the time to create logic like this if I overwhelmed myself with many difficult objects.

To navigate the scene, users should utilize the keyboard and mouse for the inputs. Moving the mouse rotates the view, so the camera acts as the viewer's eyes. To move in the space, the "WASD" keys on the keyboard are used. "W" moves the camera forward, "S" moves it backward, "Q" moves it up, "E" moves it down, "A" moves it left, and "D" moves it right. Scrolling the mouse wheel or sliding two fingers up and down on a trackpad, raises or lowers the movement speed. The keyboard pressing events are handled by the function ProcessKeyboardEvents(), which runs every frame from inside PrepareSceneView() and uses glfwGetKey() to check if any of the movement keys are pressed. Users can also press the "O"

and "P" keys to switch from orthographic to perspective projection. This is handled the same way in ProcessKeyboardEvents(). Mouse movement and mouse scrolls are handled by ProcessMouseMovement() and ProcessMouseScroll() respectively. Mouse_position_callback() handles the mouse movement and calls process mouse movement with the xoffset and yoffset calculated by subtracting the last mouse position from the new one. Mouse_scroll_wheel_callback() handles the mouse scroll by passing the scroll amount to ProcessMouseScroll() which changes the movement speed.

The helper functions I added consist of one function for each 3D object in the scene. These functions are, DrawLightstick(), DrawNotebook(), DrawCandle(), DrawCat(), DrawCamera(), each one builds its object from several basic meshes. At first, I wrote every mesh directly inside of RenderScene(), but it ended up making the function extensively long and not modular. Separating my objects into functions made it easier to read and edit RenderScene() as it became essentially a list of the objects in the scene. These functions are also reusable as they take in the objects base position, and their rotation around the Y-axis (camera and notebook only, as I chose for those to be rotated). I store repeated values such as the object's height, radius, and label position in local variables at the top. To resize or move the object, I change those variables once instead of editing the position numbers of every mesh. I also wrote a helper function, OffsetAroundY(), that takes an object’s base position and a part’s offset and rotates that offset around the Y-axis, so every part of an object stays lined up and turns together as one piece.

Overall, building the scene around a few objects of varying complexity allowed me to practice materials, textures, lighting, and camera control while recreating a real corner of my room. Moving each object into its own reusable function made my code shorter, easier to read, and easier to modify, which is the habit I’ll continue to carry into larger graphics/coding projects.